

Task 1: Create a VPC in N. Virginia

Region: US East (N. Virginia) — us-east-1

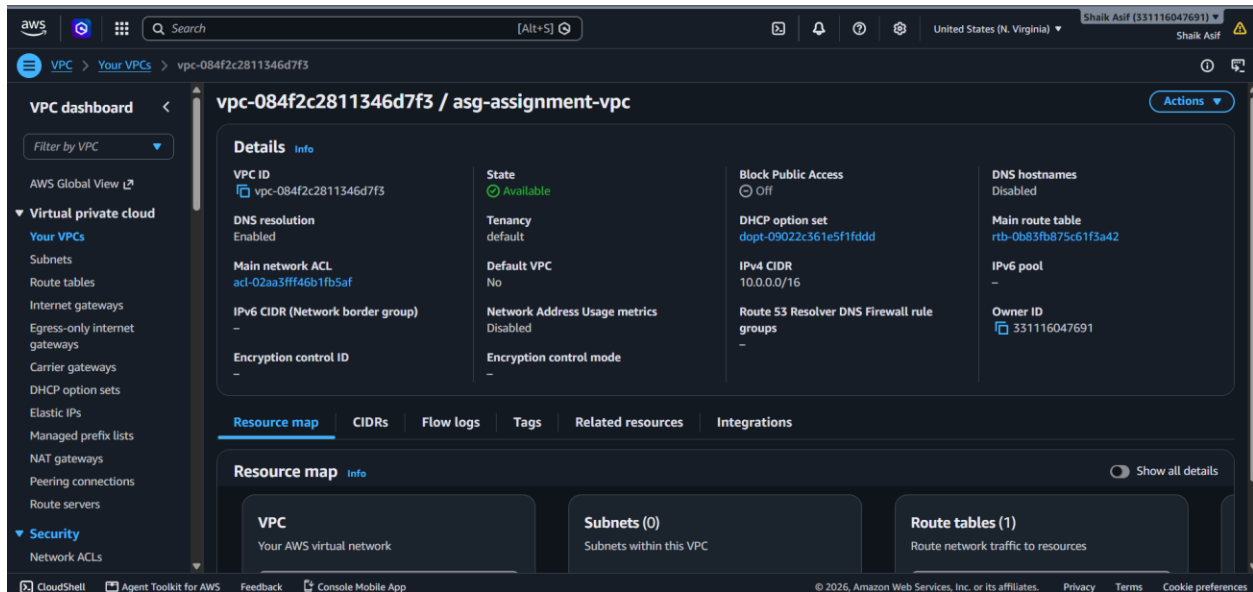
Steps used:

1. Signed in to the AWS Management Console.
2. Selected US East (N. Virginia) from the Region selector.
3. Opened the VPC service.
4. Went to Your VPCs from the left navigation pane.
5. Clicked Create VPC.
6. Selected VPC only.
7. Entered the following configuration:
 - a. Name tag: asg-assignment-vpc
 - b. IPv4 CIDR block: 10.0.0.0/16
 - c. IPv6 CIDR block: No IPv6 CIDR block
 - d. Tenancy: Default
8. Clicked Create VPC.
9. Verified that the VPC state showed Available and that the IPv4 CIDR was 10.0.0.0/16.

The screenshot displays the AWS Management Console interface for creating a new VPC. The top navigation bar shows the AWS logo, a search bar, and the current region as 'United States (N. Virginia)'. The left-hand navigation pane is open, showing the 'VPC' service and the 'Your VPCs' section. The main content area is titled 'Create VPC' and includes a brief description: 'A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.' Below this, the 'VPC settings' section is visible, which includes the following configuration options:

- Resources to create:** Two radio buttons are present. 'VPC only' is selected, while 'VPC and more' is unselected.
- Name tag - optional:** A text input field contains the value 'asg-assignment-vpc'.
- IPv4 CIDR block:** Two radio buttons are present. 'IPv4 CIDR manual input' is selected, while 'IPAM-allocated IPv4 CIDR block' is unselected. Below this, a text input field contains the value '10.0.0.0/16'.
- IPv6 CIDR block:** Four radio buttons are present. 'No IPv6 CIDR block' is selected, while 'IPAM-allocated IPv6 CIDR block', 'Amazon-provided IPv6 CIDR block', and 'IPv6 CIDR owned by me' are unselected.

At the bottom of the 'IPv4 CIDR block' section, a small note states: 'CIDR block size must be between /16 and /28.'



Conclusion:

The VPC was successfully created in the N. Virginia (us-east-1) Region with the CIDR block 10.0.0.0/16. This VPC will be used to create the public and private subnets, Internet Gateway, route tables, NAT Gateway, EC2 instances, Application Load Balancer, and Auto Scaling Group for the assignment.

Task 2: Create two subnets: one public subnet and one private subnet.

VPC used: asg-assignment-vpc

Region: US East (N. Virginia) — us-east-1

Steps used:

1. Opened the VPC service in the AWS Management Console.
2. Selected Subnets from the left navigation menu.
3. Clicked Create subnet and selected the VPC named asg-assignment-vpc.
4. Created the public subnet with the following configuration:
 - a. Subnet name: asg-public-subnet-1
 - b. Availability Zone: us-east-1a
 - c. IPv4 CIDR block: 10.0.1.0/24
5. Created the subnet and selected it from the Subnets list.
6. Opened Actions → Edit subnet settings.

7. Enabled Auto-assign public IPv4 address and saved the change.
8. Clicked Create subnet again and selected asg-assignment-vpc.
9. Created the private subnet with the following configuration:
 - a. Subnet name: asg-private-subnet-1
 - b. Availability Zone: us-east-1b
 - c. IPv4 CIDR block: 10.0.2.0/24
10. Left the Auto-assign public IPv4 address option disabled for the private subnet.
11. Verified both subnets in the Subnets list.

subnets > subnet-0078533664600cfe

subnet-0078533664600cfe / asg-public-subnet-1

Details

Subnet ID subnet-0078533664600cfe	Subnet ARN arn:aws:ec2:us-east-1:331116047691:subnet/subnet-0078533664600cfe	State Available	Block Public Access Off
IPv4 CIDR 10.0.1.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone us-east-1a	Network border group us-east-1	VPC vpc-084f2c2811346d7f3 asg-assignment-vpc	Route table rtb-0b83fb875c61f3a42
Network ACL acl-02aa3fff46b1fb5af	Default subnet No	Auto-assign public IPv4 address Yes	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record Disabled	DNS64 Disabled	Owner 331116047691	

Flow logs | **Route table** | **Network ACL** | **CIDR reservations** | **Sharing** | **Tags** | **Related resources**

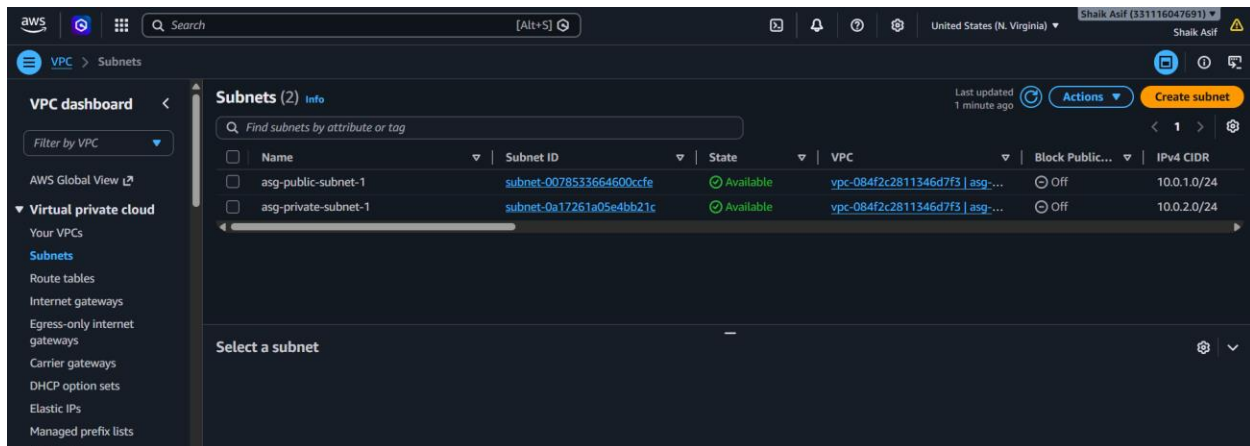
subnets > subnet-0a17261a05e4bb21c

subnet-0a17261a05e4bb21c / asg-private-subnet-1

Details

Subnet ID subnet-0a17261a05e4bb21c	Subnet ARN arn:aws:ec2:us-east-1:331116047691:subnet/subnet-0a17261a05e4bb21c	State Available	Block Public Access Off
IPv4 CIDR 10.0.2.0/24	Available IPv4 addresses 251	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone us-east-1b	Network border group us-east-1	VPC vpc-084f2c2811346d7f3 asg-assignment-vpc	Route table rtb-0b83fb875c61f3a42
Network ACL acl-02aa3fff46b1fb5af	Default subnet No	Auto-assign public IPv4 address No	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record Disabled	DNS64 Disabled	Owner 331116047691	

Flow logs | **Route table** | **Network ACL** | **CIDR reservations** | **Sharing** | **Tags** | **Related resources**



Conclusion:

The public subnet asg-public-subnet-1 with CIDR block 10.0.1.0/24 and the private subnet asg-private-subnet-1 with CIDR block 10.0.2.0/24 were successfully created inside asg-assignment-vpc. The public subnet is configured to assign public IPv4 addresses automatically, while the private subnet remains isolated from direct public IP assignment. These subnets will be used for the NAT Gateway and EC2 instances in the next tasks.

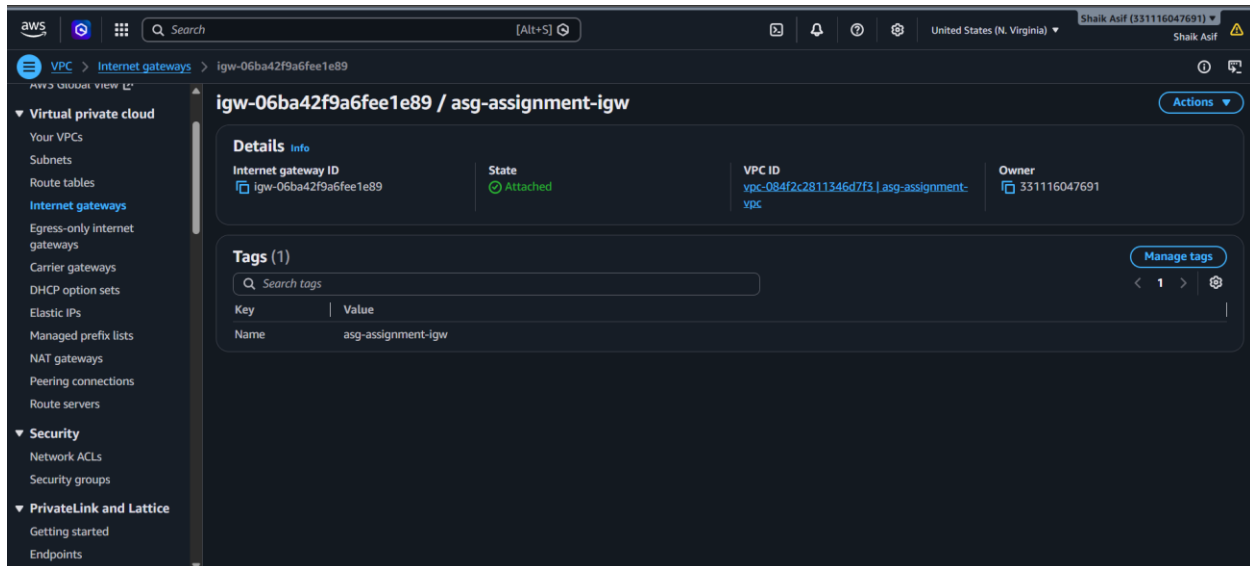
Task 3: Create and Attach an Internet Gateway

VPC used: asg-assignment-vpc

Region: US East (N. Virginia) — us-east-1

Steps used:

1. Opened the VPC service in the AWS Management Console.
2. Selected Internet gateways from the left navigation menu.
3. Clicked Create internet gateway.
4. Entered the name tag asg-assignment-igw.
5. Clicked Create internet gateway.
6. Selected the newly created Internet Gateway.
7. Clicked Actions → Attach to VPC.
8. Selected the VPC named asg-assignment-vpc.
9. Clicked Attach internet gateway.
10. Verified that the Internet Gateway state changed to Attached.



Conclusion:

The Internet Gateway asg-assignment-igw was successfully created and attached to asg-assignment-vpc. It provides the VPC with a connection point to the internet. Internet access will become available to resources in the public subnet after a public route table is configured with a route to this Internet Gateway.

Task 4: Create one public route table (RT) and one private route table.

VPC used: asg-assignment-vpc

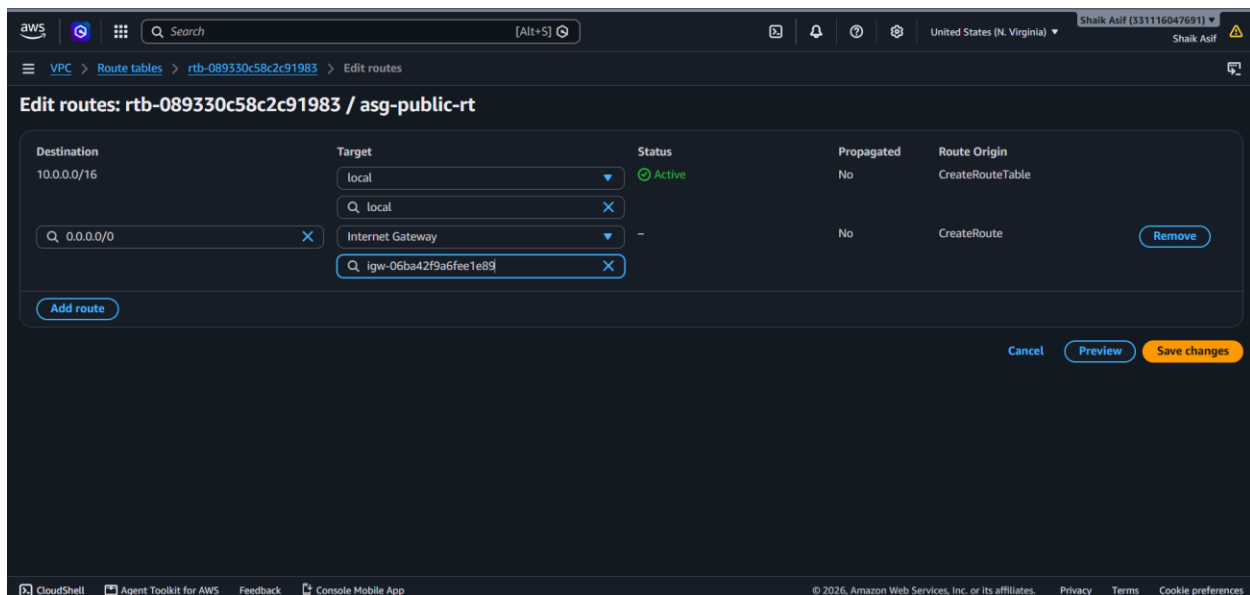
Internet Gateway used: asg-assignment-igw

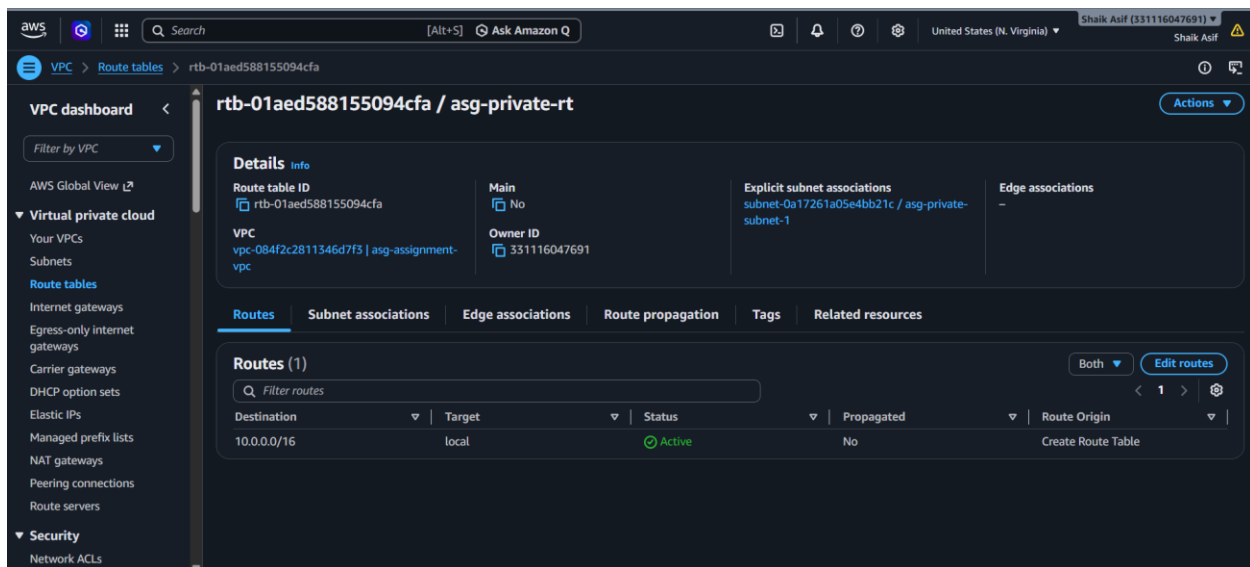
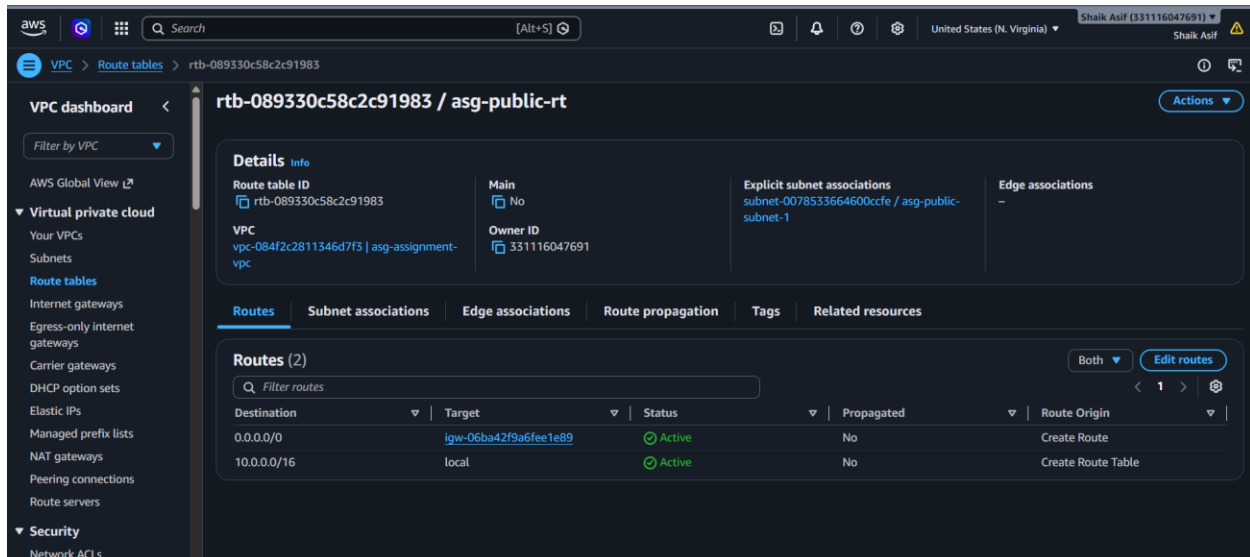
Region: US East (N. Virginia) — us-east-1

Steps used:

1. Opened the VPC service in the AWS Management Console.
2. Selected Route tables from the left navigation menu.
3. Clicked Create route table.
4. Created the public route table with the following configuration:
 - a. Name: asg-public-rt
 - b. VPC: asg-assignment-vpc
5. Selected asg-public-rt and opened the Routes tab.
6. Clicked Edit routes → Add route.
7. Added the following public internet route:
 - a. Destination: 0.0.0.0/0

- b. Target: Internet Gateway asg-assignment-igw
8. Saved the route changes.
9. Opened the Subnet associations tab for asg-public-rt.
10. Associated asg-public-subnet-1 (10.0.1.0/24) with the public route table.
11. Clicked Create route table again.
12. Created the private route table with the following configuration:
 - Name: asg-private-rt
 - VPC: asg-assignment-vpc
13. Opened the Subnet associations tab for asg-private-rt.
14. Associated asg-private-subnet-1 (10.0.2.0/24) with the private route table.
15. Verified that the private route table contained only the default local route (10.0.0.0/16 → local) at this stage.





Conclusion:

The public and private route tables were successfully created and associated with their respective subnets. The public subnet now has internet access through the Internet Gateway using the route 0.0.0.0/0 → asg-assignment-igw. The private subnet remains isolated from direct internet access and will receive outbound internet connectivity through a NAT Gateway in the next task.

Task 5: Create a NAT Gateway and Configure Private Subnet Internet Access

VPC used: asg-assignment-vpc

Public subnet used: asg-public-subnet-1

Private route table used: asg-private-rt

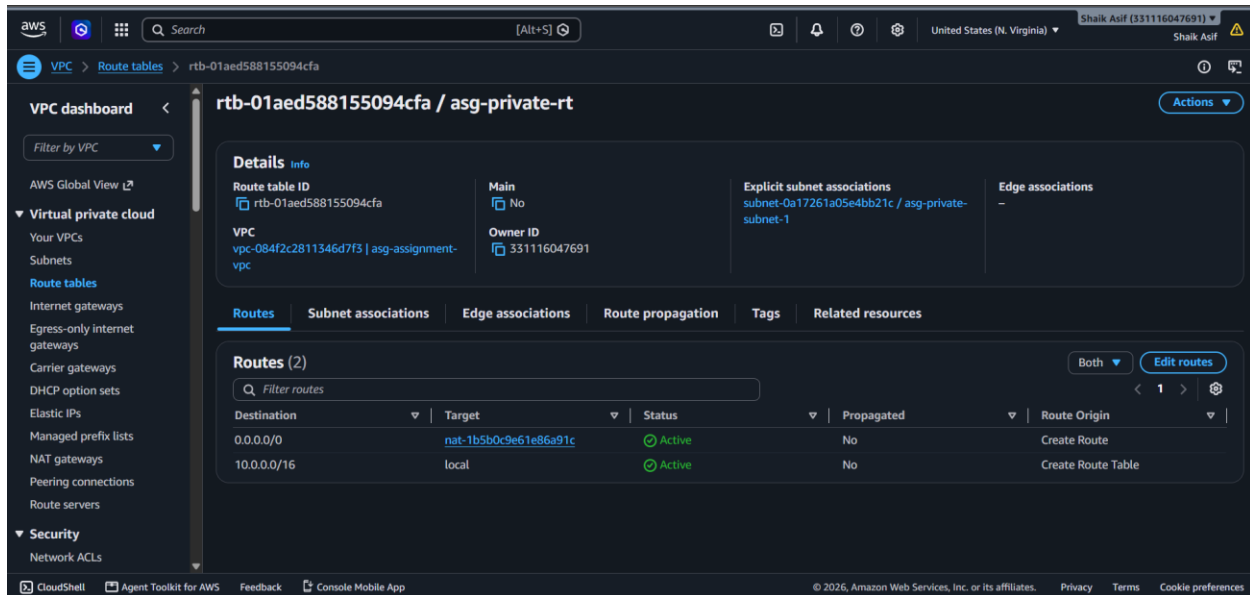
Region: US East (N. Virginia) — us-east-1

Steps used:

1. Opened the VPC service in the AWS Management Console.
2. Selected Elastic IPs from the left navigation menu.
3. Clicked Allocate Elastic IP address and kept the default settings.
4. Clicked Allocate to create a new Elastic IP address.
5. Selected NAT gateways from the left navigation menu.
6. Clicked Create NAT gateway.
7. Entered the following NAT Gateway configuration:
 - a. Name: asg-nat-gateway
 - b. Subnet: asg-public-subnet-1 (10.0.1.0/24)
 - c. Connectivity type: Public
 - d. Elastic IP allocation ID: Selected the newly allocated Elastic IP address
8. Clicked Create NAT gateway.
9. Waited until the NAT Gateway state changed to Available.
10. Opened Route tables and selected asg-private-rt.
11. Opened the Routes tab and clicked Edit routes.
12. Added the following route:
 - Destination: 0.0.0.0/0
 - Target: NAT Gateway asg-nat-gateway
13. Saved the route changes.
14. Verified that the public route table still routed 0.0.0.0/0 through asg-assignment-igw.

Evidence captured:

- Screenshot of the NAT Gateway details page showing its name, state as Available, public subnet, and associated Elastic IP address.
- Screenshot of asg-private-rt routes showing 0.0.0.0/0 routed to asg-nat-gateway.



Conclusion:

The NAT Gateway asg-nat-gateway was successfully created in asg-public-subnet-1 and assigned an Elastic IP address. The private route table was updated so that outbound traffic from asg-private-subnet-1 is routed through the NAT Gateway. This allows private resources to access the internet for updates and software installation while remaining inaccessible directly from the internet.

Task 6: Create two instances, one in the public subnet and one in the private subnet.

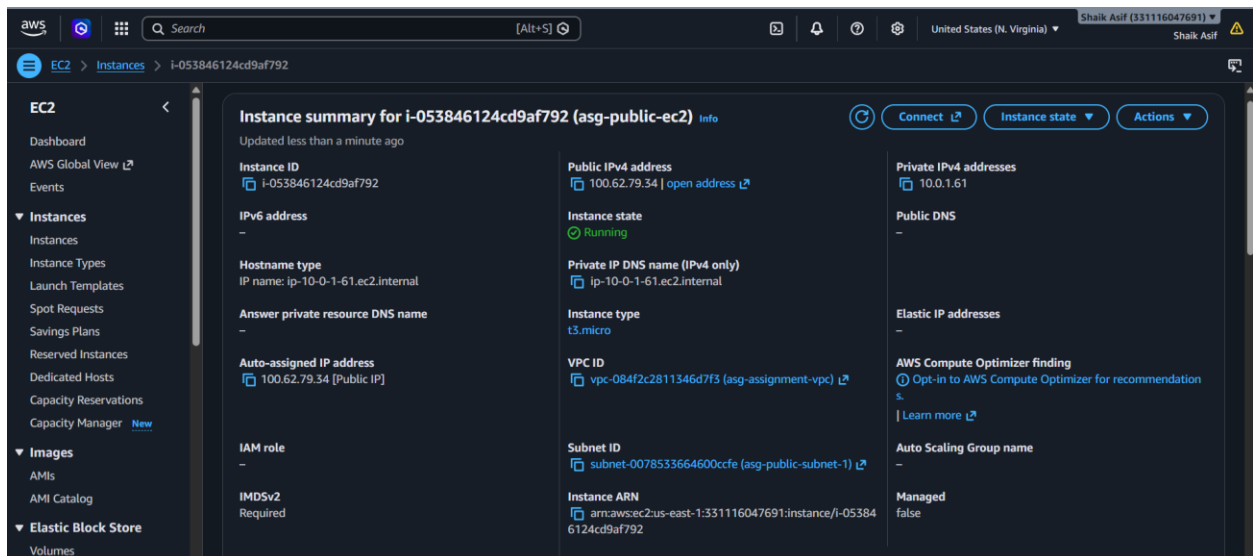
VPC used: asg-assignment-vpc

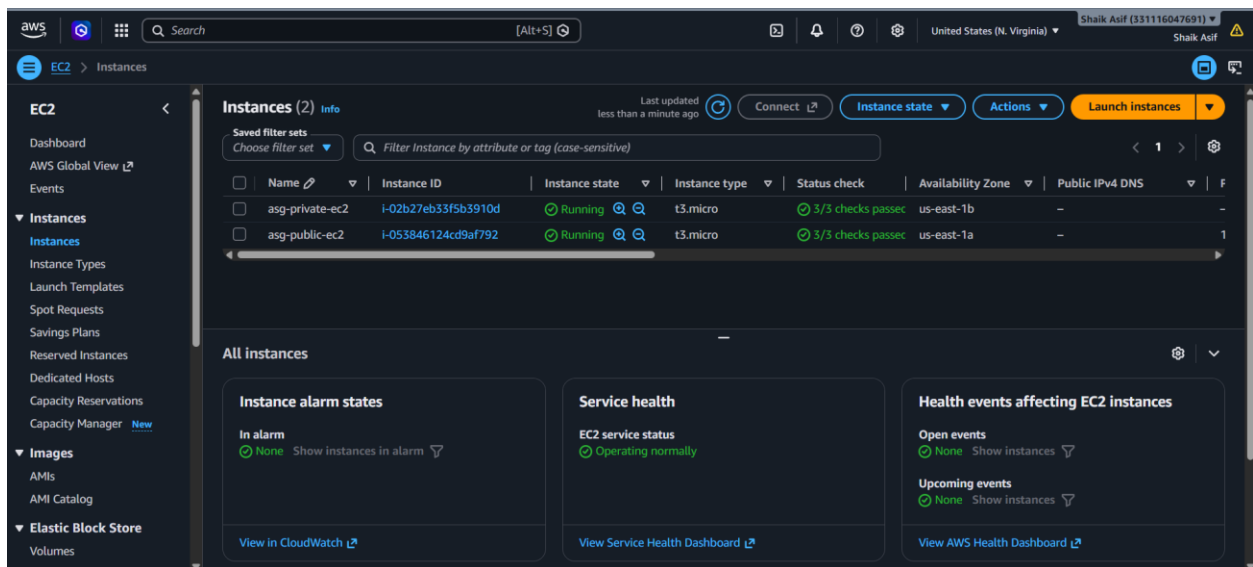
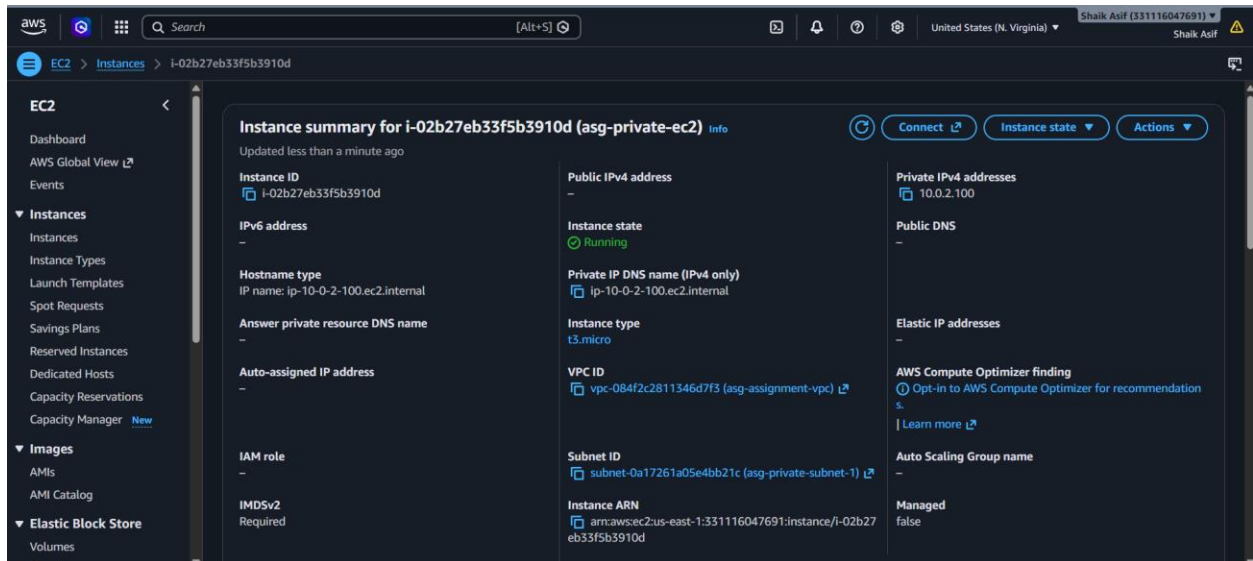
Region: US East (N. Virginia) — us-east-1

Steps used:

1. Opened the EC2 service in the AWS Management Console.
2. Created an EC2 key pair named asg-assignment-key using the RSA key type and .pem private key format.
3. Selected Instances and clicked Launch instances.

4. Created the public EC2 instance with the following configuration:
 - a. Name: asg-public-ec2
 - b. AMI: Amazon Linux 2023
 - c. Instance type: t2.micro or t3.micro
 - d. Key pair: asg-assignment-key
 - e. VPC: asg-assignment-vpc
 - f. Subnet: asg-public-subnet-1
 - g. Auto-assign public IP: Enabled
 - h. Security group: asg-public-ec2-sg
5. Configured the public instance security group to allow SSH access on port 22 only from My IP.
6. Left the User data field empty and launched the public instance.
7. Clicked Launch instances again.
8. Created the private EC2 instance with the following configuration:
 - a. Name: asg-private-ec2
 - b. AMI: Amazon Linux 2023
 - c. Instance type: t2.micro or t3.micro
 - d. Key pair: asg-assignment-key
 - e. VPC: asg-assignment-vpc
 - f. Subnet: asg-private-subnet-1
 - g. Auto-assign public IP: Disabled
 - h. Security group: asg-private-ec2-sg
9. Kept the User data field empty and launched the private instance.
10. Verified that both instances were in the Running state and had passed the EC2 status checks.





Conclusion:

Two EC2 instances were successfully created in separate subnets within asg-assignment-vpc. The public instance, asg-public-ec2, was launched in asg-public-subnet-1 with a public IPv4 address. The private instance, asg-private-ec2, was launched in asg-private-subnet-1 without a public IPv4 address.

Task 7: Deploy Apache server on both EC2 instances with a sample index.html file.

Instances used: asg-public-ec2 and asg-private-ec2

Region: US East (N. Virginia) — us-east-1

Steps used:

1. Connected to asg-public-ec2 from Git Bash using the downloaded key pair asg-assignment-key.pem.
2. Installed Apache HTTP Server on the public EC2 instance using:

```
sudo dnf install -y httpd
```

3. Enabled and started the Apache service:

```
sudo systemctl enable --now httpd
```

4. Created a sample web page on the public instance:

```
echo '<h1>Apache Server - Public EC2 Instance</h1>' | sudo tee /var/www/html/index.html
```

5. Verified the public Apache service by accessing the public instance's public IPv4 address in a web browser.
6. Updated the private instance security group to allow SSH on port 22 only from asg-public-ec2-sg.
7. Created an SSH tunnel from the local computer through the public EC2 instance to the private EC2 instance on port 22.
8. Connected to asg-private-ec2 through the SSH tunnel.

9. Installed Apache HTTP Server on the private EC2 instance:

```
sudo dnf install -y httpd
```

10. Enabled and started Apache:

```
sudo systemctl enable --now httpd
```

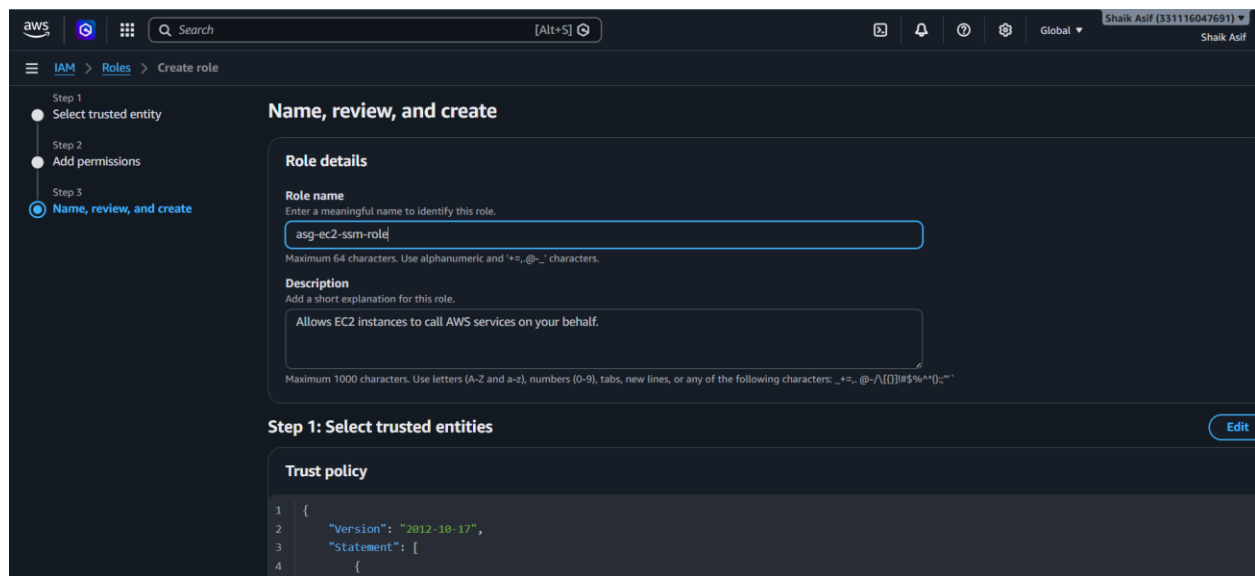
11. Created a sample web page on the private instance:

```
echo '<h1>Apache Server - Private EC2 Instance</h1>' | sudo tee /var/www/html/index.html
```

12. Verified Apache locally on the private instance using:

```
curl http://localhost
```

13. Created a second SSH tunnel that forwarded local port 8080 through the public EC2 instance to private instance port 80.



Search

[Alt+S]

Global

Shaik Asif (331116047691)

Identity and Access Management (IAM)

Dashboard

Access Management

- Roles
- Policies
- IAM users
- IAM user groups
- Identity providers
- Account settings
- Root access management
- Temporary delegation requests

Access reports

- Access Analyzer
 - Resource analysis
 - Unused access
 - Analyzer settings
- Policy simulator

asg-ec2-ssm-role

Info

Delete

Allows EC2 instances to call AWS services on your behalf.

Summary

Edit

Creation date

September 11, 2026, 11:45 (UTC+05:30)

ARN

arn:aws:iam::331116047691:role/asg-ec2-ssm-role

Instance profile ARN

arn:aws:iam::331116047691:instance-profile/asg-ec2-ssm-role

Last activity

-

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (1)

Info

Simulate

Remove

Add permissions

You can attach up to 20 managed policies.

Filter by Type

Search

All types

< 1 >

Policy name

Type

Attached entities

AmazonSSMManagedInstanceCore

AWS managed

1

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (331116047691)

EC2

Instances

i-053846124cd9af792

EC2

Dashboard

AWS Global View

Events

Instances

- Instances
- Instance Types
- Launch Templates
- Spot Requests
- Savings Plans
- Reserved Instances
- Dedicated Hosts
- Capacity Reservations
- Capacity Manager

Images

- AMIs
- AMI Catalog

Elastic Block Store

- Volumes

Instance summary for i-053846124cd9af792 (asg-public-ec2)

Info

Connect

Instance state

Actions

Updated less than a minute ago

Instance ID

i-053846124cd9af792

Public IPv4 address

100.62.79.34 | open address

Private IPv4 addresses

10.0.1.61

IPv6 address

-

Instance state

Running

Public DNS

-

Hostname type

IP name: ip-10-0-1-61.ec2.internal

Private IP DNS name (IPv4 only)

ip-10-0-1-61.ec2.internal

Elastic IP addresses

-

Answer private resource DNS name

-

Instance type

t3.micro

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendation

Learn more

Auto-assigned IP address

100.62.79.34 [Public IP]

VPC ID

vpc-084f2c2811346d7f3 (asg-assignment-vpc)

Auto Scaling Group name

-

IAM role

asg-ec2-ssm-role

Subnet ID

subnet-0078533664600ccfe (asg-public-subnet-1)

Managed

false

IMDSv2

Required

Instance ARN

arn:aws:ec2:us-east-1:331116047691:instance/i-053846124cd9af792

Operator

-

AWS Management Console screenshot showing the EC2 Instance summary for i-02b27eb33f5b3910d (asg-private-ec2).

Instance summary for i-02b27eb33f5b3910d (asg-private-ec2)

Updated less than a minute ago

Instance ID: i-02b27eb33f5b3910d

IPv6 address: -

IPV6 address: -

Hostname type: IP name: ip-10-0-2-100.ec2.internal

Answer private resource DNS name: -

Auto-assigned IP address: -

IAM role: asg-ec2-ssm-role

IMDSv2: Required

Public IPv4 address: -

Instance state: Running

Private IP DNS name (IPv4 only): ip-10-0-2-100.ec2.internal

Instance type: t3.micro

VPC ID: vpc-084f2c2811346d7f3 (asg-assignment-vpc)

Subnet ID: subnet-0a17261a05e4bb21c (asg-private-subnet-1)

Instance ARN: arn:aws:ec2:us-east-1:331116047691:instance/i-02b27eb33f5b3910d

Private IPv4 addresses: 10.0.2.100

Public DNS: -

Elastic IP addresses: -

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendation. Learn more

Auto Scaling Group name: -

Managed: false

```
ec2-user@ip-10-0-1-61:~$ ssh -i asg-assignment-key.pem ec2-user@100.62.79.34
Warning: Permanently added '100.62.79.34' (ED25519) to the list of known hosts.
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
```

```
ec2-user@ip-10-0-1-61:~$ sudo dnf install -y httpd
Amazon Linux 2023 kernel Livepatch repository
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
httpd x86_64 2.4.68-1.amzn2023.0.1 amazonlinux 46 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
apr-util x86_64 1.6.5-1.amzn2023.0.1 amazonlinux 100 k
apr-util-ldap x86_64 1.6.5-1.amzn2023.0.1 amazonlinux 15 k
generic-logos-httpd noarch 18.0.0-12.amzn2023.0.3 amazonlinux 19 k
httpd-core x86_64 2.4.68-1.amzn2023.0.1 amazonlinux 1.4 M
httpd-filesystem noarch 2.4.68-1.amzn2023.0.1 amazonlinux 12 k
httpd-tools x86_64 2.4.68-1.amzn2023.0.1 amazonlinux 80 k
libbrotli x86_64 1.0.9-4.amzn2023.0.2 amazonlinux 315 k
mailcap noarch 2.1.49-3.amzn2023.0.3 amazonlinux 33 k
Installing weak dependencies:
apr-util-openssl x86_64 1.6.5-1.amzn2023.0.1 amazonlinux 17 k
mod_http2 x86_64 2.0.42-1.amzn2023.0.1 amazonlinux 167 k
mod_lua x86_64 2.4.68-1.amzn2023.0.1 amazonlinux 59 k
Transaction Summary
-----
Install 13 Packages
Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm 3.3 MB/s | 129 kB 00:00
(2/13): apr-util-ldap-1.6.5-1.amzn2023.0.1.x86_64.rpm 35.5 MB/s | 15 kB 00:00
(3/13): apr-util-1.6.5-1.amzn2023.0.1.x86_64.rpm 2.0 MB/s | 100 kB 00:00
(4/13): apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64.rpm 655 kB/s | 17 kB 00:00
(5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm 638 kB/s | 19 kB 00:00
(6/13): httpd-2.4.68-1.amzn2023.0.1.x86_64.rpm 1.6 MB/s | 46 kB 00:00
(7/13): httpd-filesystem-2.4.68-1.amzn2023.0.1.noarch.rpm 520 kB/s | 12 kB 00:00
(8/13): httpd-core-2.4.68-1.amzn2023.0.1.x86_64.rpm 28 MB/s | 1.4 MB 00:00
(9/13): httpd-tools-2.4.68-1.amzn2023.0.1.x86_64.rpm 1.9 MB/s | 80 kB 00:00
(10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm 9.3 MB/s | 315 kB 00:00
(11/13): mailcap-2.1.49-3.amzn2023.0.3.noarch.rpm 1.2 MB/s | 33 kB 00:00
(12/13): mod_http2-2.0.42-1.amzn2023.0.1.x86_64.rpm 5.0 MB/s | 167 kB 00:00
(13/13): mod_lua-2.4.68-1.amzn2023.0.1.x86_64.rpm 2.0 MB/s | 59 kB 00:00
Total 12 MB/s | 2.4 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing:
Installing:
: apr-1.7.5-1.amzn2023.0.4.x86_64 1/13
Installing:
: apr-util-ldap-1.6.5-1.amzn2023.0.1.x86_64 2/13
Installing:
: apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64 3/13
Installing:
: apr-util-1.6.5-1.amzn2023.0.1.x86_64 4/13
Installing:
: mailcap-2.1.49-3.amzn2023.0.3.noarch 5/13
Installing:
: httpd-tools-2.4.68-1.amzn2023.0.1.x86_64 6/13
Installing:
: libbrotli-1.0.9-4.amzn2023.0.2.x86_64 7/13
```

```
ec2-user@ip-10-0-1-61:~$ sudo systemctl enable --now httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
ec2-user@ip-10-0-1-61:~$ echo '<h1>Apache Server - Public EC2 Instance</h1>' | sudo tee /var/www/html/index.html
<h1>Apache Server - Public EC2 Instance</h1>
```

← → ↻ Not secure 100.62.79.34 ☆ 📄 📁 🌐 ⋮

Apache Server - Public EC2 Instance

[illegible]

Steps

1. Create Application Load Balancer

- Open AWS Console → EC2 → Load Balancers.
- Click Create Load Balancer.
- Select Application Load Balancer.
- Name: asg-application-alb
- Scheme: Internet-facing
- IP address type: IPv4.
- Select the required VPC.
- Select two public subnets:
 - us-east-1a
 - us-east-1b

2. Configure ALB Security Group

Created security group:

asg-alb-sg

Inbound rule:

- HTTP – TCP 80
- Source: 0.0.0.0/0

Outbound:

- Allow all traffic.

This allows users from the internet to access the ALB through HTTP.

3. Create Target Group

- Go to EC2 → Target Groups.
- Click Create target group.
- Target type: Instances
- Name: asg-target-group
- Protocol: HTTP
- Port: 80
- Health check protocol: HTTP

- Health check path: /
- Register the EC2 instances as targets.

4. Configure ALB Listener

Created listener:

- Protocol: HTTP
- Port: 80
- Default action: Forward to asg-target-group

The ALB receives requests on port 80 and forwards them to the registered EC2 instances.

5. Configure EC2 Security Group

The EC2 security group was configured as:

asg-public-ec2-sg

Inbound rules:

- SSH – TCP 22 → My IP address
- HTTP – TCP 80 → Source: ALB security group asg-alb-sg

The HTTP rule was changed from public access to the ALB security group.

This ensures that the EC2 servers accept HTTP traffic from the ALB rather than directly from everyone on the internet.

6. Verify Target Health

- Open Target Groups → asg-target-group → Targets.
- Verified that the registered EC2 instances became:

Healthy

This confirmed that the ALB could communicate with the EC2 instances.

7. Test ALB DNS

Copied the ALB DNS name:

asg-application-alb-2118538500.us-east-1.elb.amazonaws.com

Opened it in a browser using:

[HTTP://ALB-DNS-NAME](http://ALB-DNS-NAME)

The Apache web page/application was successfully displayed.

Traffic Flow

Internet → ALB → Target Group → EC2 → Apache

The screenshot shows the 'Create security group' page in the AWS Management Console. The page is for the 'us-east-1' region. The 'Basic details' section is filled out: 'Security group name' is 'asg-alb-sg', 'Description' is 'Security group for Application Load Balancer', and 'VPC' is 'vpc-03c93e7fbbfe946e6 (asg-assignment-vpc)'. The 'Inbound rules' section shows a rule for 'HTTP' on 'TCP' port '80' from 'Anywh...' with a source of '0.0.0.0/0'. There is an 'Add rule' button at the bottom left of the rules section.

The screenshot shows the 'Create target group' page in the AWS Management Console. The page is for the 'us-east-1' region. The 'Settings - immutable' section is visible. The 'Target type' is set to 'Instances'. The 'Target group name' is 'asg-target-group'. The page includes a progress bar on the left with steps: 'Step 1: Create target group' (selected), 'Step 2 - recommended: Register targets', and 'Step 3: Review and create'.

EC2

Target groups

asg-target-group

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

EBS Snapshots

asg-target-group

Actions

Details

arn:aws:elasticloadbalancing:us-east-1:331116047691:targetgroup/asg-target-group/99537a2294368efd

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

vpc-03c93e7fbbfe946e6

IP address type

IPv4

Load balancer

None associated

2

Total targets

0

Healthy

0

Unhealthy

2

Unused

0

Initial

0

Draining

0

Anomalous

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Health check settings

Edit

Protocol

Path

Port

Healthy threshold

EC2

Load balancers

Create Application Load Balancer

How Application Load Balancers work

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

asg-application-alb

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme

Info

Scheme can't be changed after the load balancer is created.

Internet-facing

Serves internet-facing traffic.

Has public IP addresses.

DNS name resolves to public IPs.

Requires a public subnet.

Internal

Serves internal traffic.

Has private IP addresses.

DNS name resolves to private IPs.

Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type

Info

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

IPv4

Includes only IPv4 addresses.

Dualstack

EC2 > Load balancers > asg-application-alb

Elastic Block Store

Network & Security

Load Balancing

Auto Scaling

Settings

Volumes

Snapshots

Lifecycle Manager

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancers

Target Groups

Trust Stores

Auto Scaling Groups

asg-application-alb

Details

Load balancer type

Application

Status

Active

VPC

vpc-03c93e7fbbfe946e6

Load balancer IP address type

IPv4

Scheme

Internet-facing

Hosted zone

Z355XD0TRQ7X7K

Availability Zones

subnet-001a5f5e73240ad3a us-east-1a (use1-az1)

subnet-0bf21e57763d42a26 us-east-1b (use1-az2)

Date created

September 15, 2026, 18:02 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:us-east-1:331116047691:loadbalancer/app/asg-application-alb/88d1f174ab26bd51

DNS name

asg-application-alb-2118538500.us-east-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Listeners and rules (1)

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Manage rules

Manage listener

Add listener

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

EC2 > Target groups > asg-target-group

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

Elastic Block Store

AMIs

AMI Catalog

Volumes

asg-target-group

Details

arn:aws:elasticloadbalancing:us-east-1:331116047691:targetgroup/asg-target-group/99537a2294368efd

Target type

Instance

Protocol : Port

HTTP: 80

Protocol version

HTTP1

VPC

vpc-03c93e7fbbfe946e6

IP address type

IPv4

Load balancer

asg-application-alb

2

Total targets

2

Healthy

0

Unhealthy

0

Anomalous

0

Unused

0

Initial

0

Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target

Anomaly mitigation: Not applicable

Deregister

Register targets

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



Conclusion

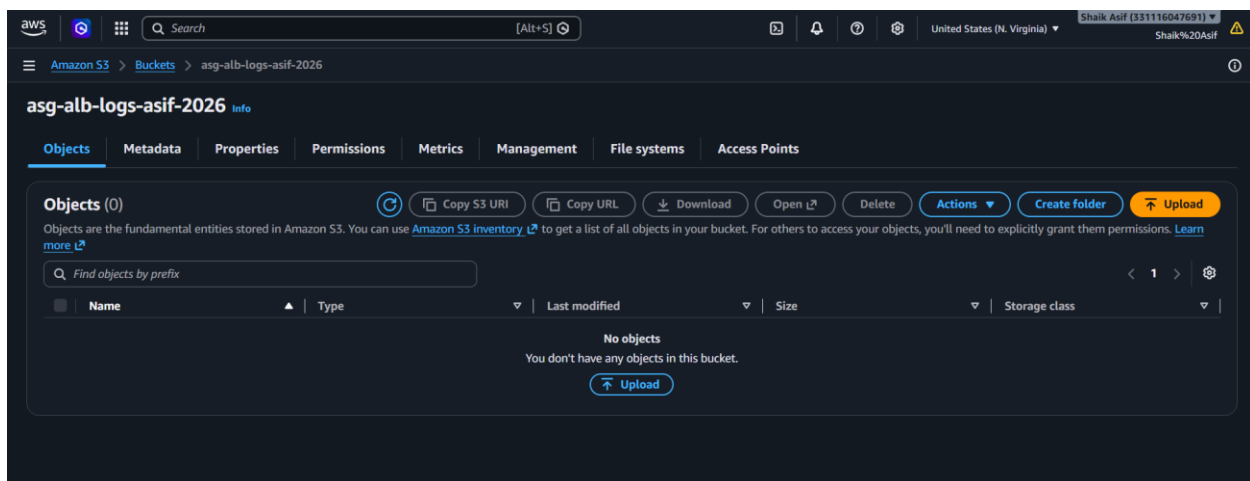
The Application Load Balancer was successfully configured and connected to the EC2 instances through a target group. The ALB listener was configured on HTTP port 80, the target instances became healthy, and the EC2 security group was restricted to accept HTTP traffic from the ALB security group.

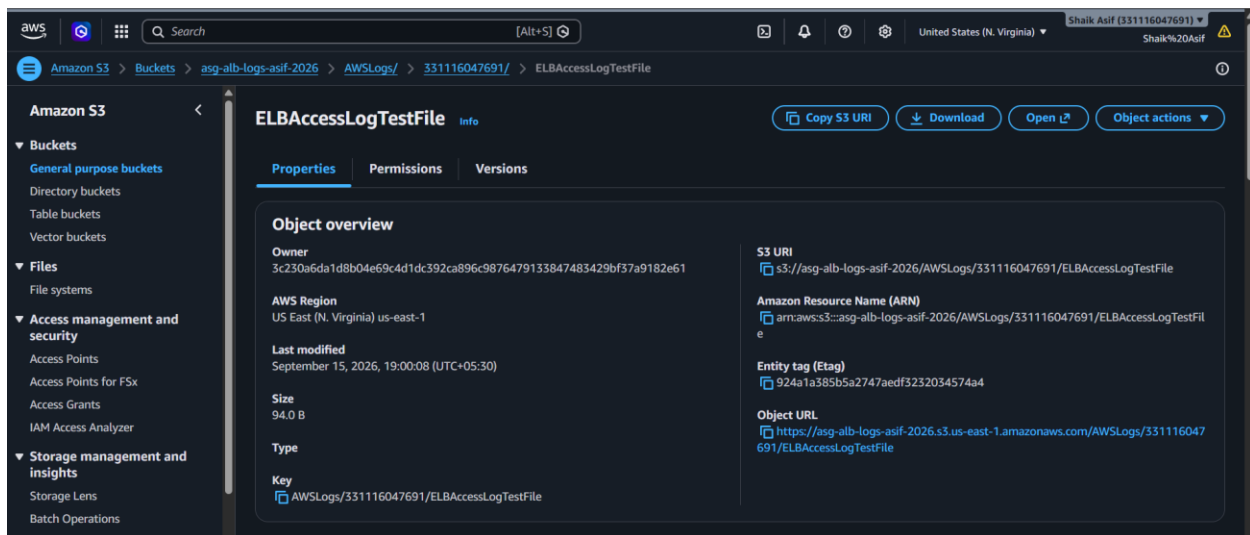
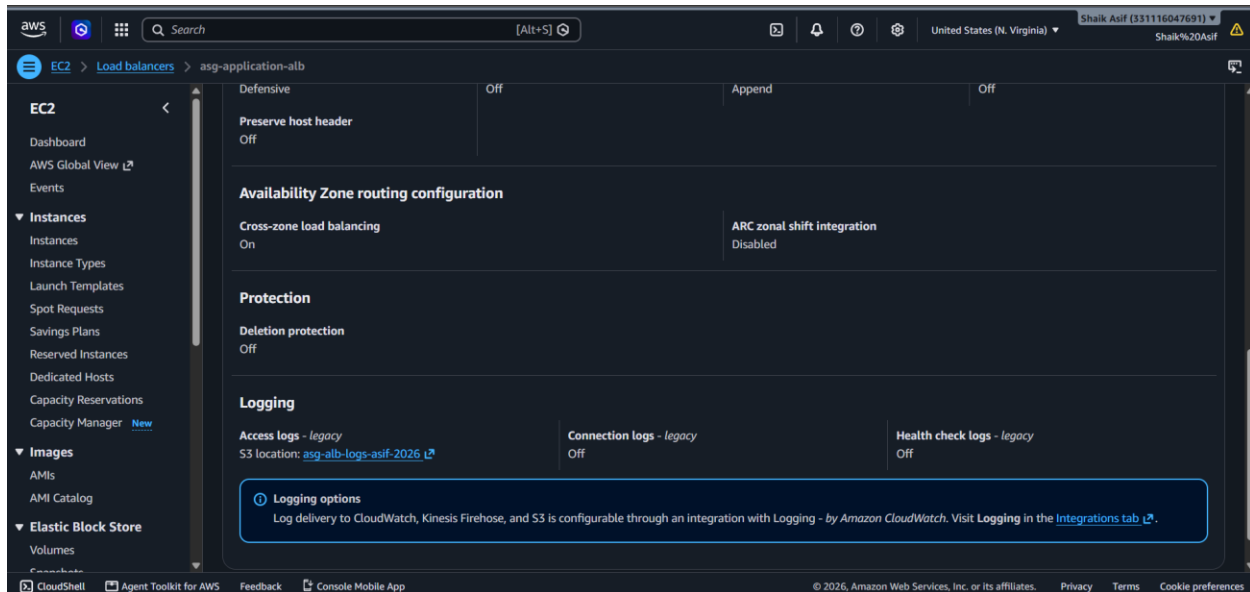
Finally, the application was successfully accessed through the ALB DNS name, confirming that the load-balancing setup was working correctly.

Task 9: Store application load balancer logs in S3.

Steps I Used

1. Opened Amazon S3 and created an S3 bucket:
 - a. asg-alb-logs-asif-2026
2. Opened the bucket and went to Permissions → Bucket Policy.
3. Added a bucket policy to allow the Elastic Load Balancing log delivery service to write access logs to the bucket.
4. Used the AWS Account ID:
 - a. 331116047691
5. Opened EC2 → Load Balancers and selected:
 - a. asg-application-alb
6. Went to Attributes → Access logs → Edit.
7. Enabled Access logs.
8. Selected the S3 bucket:
 - a. asg-alb-logs-asif-2026
9. Left the prefix empty and saved the configuration.
10. The ALB access logging configuration was successfully saved.





Conclusion

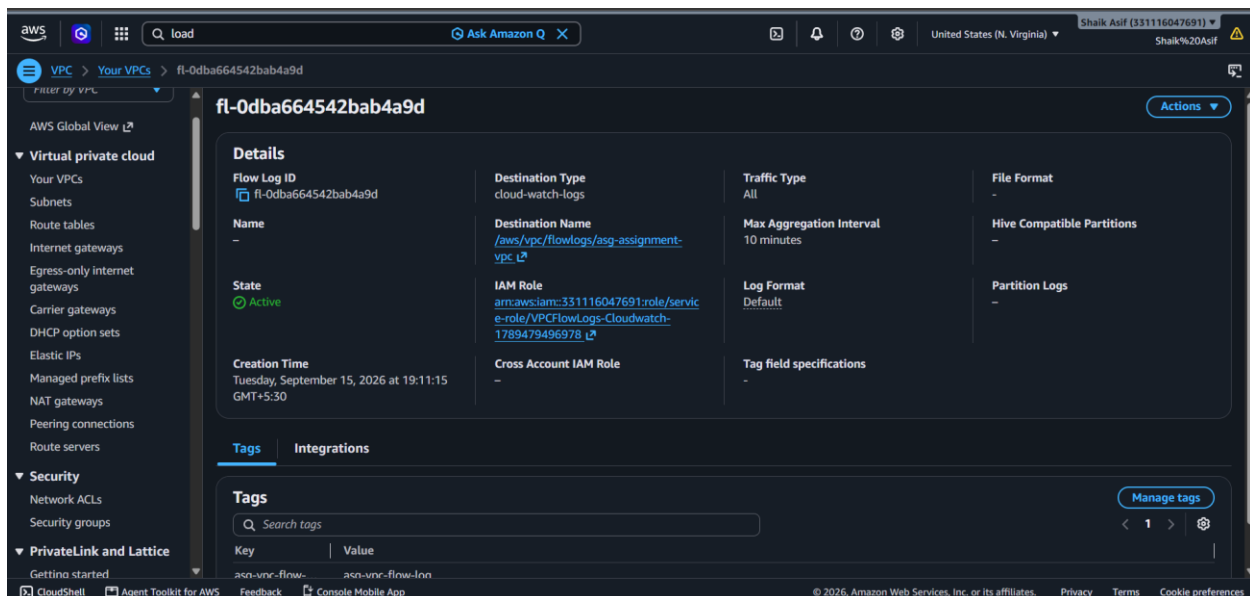
ALB access logging was successfully configured. The Application Load Balancer is now enabled to store access logs in the Amazon S3 bucket for monitoring, troubleshooting, and traffic analysis.

Task 10: Store the VPC flow logs in a CloudWatch log group.

Steps Used

1. Opened the VPC Console in the N. Virginia (us-east-1) region.
2. Selected the VPC asg-assignment-vpc.

3. Opened Flow Logs and selected Create flow log.
4. Selected All traffic to capture both accepted and rejected traffic.
5. Set the maximum aggregation interval to 10 minutes.
6. Selected Send to CloudWatch Logs as the destination.
7. Created/selected the CloudWatch log group:
/aws/vpc/flowlogs/asg-assignment-vpc
8. Selected Create and use a new service role for CloudWatch Logs access.
9. Selected the AWS default log record format.
10. Added the tag/name asg-vpc-flow-log.
11. Created the flow log successfully and verified that it became Active.



Conclusion

VPC Flow Logs were successfully configured for asg-assignment-vpc to capture all network traffic and send the flow logs to CloudWatch Logs for monitoring and troubleshooting.

Task 12: Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

Steps Used

1. Created CPU Utilization Dashboard

- Opened AWS CloudWatch in the N. Virginia (us-east-1) region.

- Created a CloudWatch Dashboard.
- Added an EC2 CPU Utilization metric/widget to monitor the CPU usage of the EC2 instance.

2. Connected to the EC2 Instance

- Connected to the asg-public-ec2 instance using SSH.
- Verified that the Apache service was running using:

```
sudo systemctl status httpd
```

- Confirmed that Apache was active/running.

3. Installed CloudWatch Agent

Installed the Amazon CloudWatch Agent on the EC2 instance:

```
sudo dnf install -y amazon-cloudwatch-agent
```

The installation completed successfully.

4. Created CloudWatch Agent Configuration

Created the CloudWatch Agent configuration file:

```
/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

Configured the agent to monitor the Apache httpd process using the procstat metric and collect the pid_count metric every 60 seconds.

The metric was configured under the custom namespace:

Apache/Monitoring

5. Corrected the Configuration

Initially, the procstat configuration format was incorrect, so the CloudWatch Agent returned a JSON schema validation error.

The configuration was corrected by changing procstat to the required array format.

After correction, the configuration validation succeeded.

6. Started the CloudWatch Agent

Started the CloudWatch Agent using its configuration.

Then checked its status:

```
sudo systemctl status amazon-cloudwatch-agent
```

The agent showed:

Active: active (running)

7. Troubleshoot Missing CloudWatch Metrics

Initially, the Apache/Monitoring namespace was not visible in CloudWatch.

Checked the CloudWatch Agent logs:

```
sudo tail -50 /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log
```

The logs showed:

NoCredentialProviders

and:

no EC2 instance role found

This indicated that the EC2 instance did not have an IAM role with permission to publish CloudWatch metrics.

8. Created IAM Role

Created an IAM role named:

instanceRole

Selected EC2 as the trusted entity/use case.

Attached the required policies:

- CloudWatchAgentServerPolicy
- AmazonSSMManagedInstanceCore

The important policy for CloudWatch monitoring was:

CloudWatchAgentServerPolicy

9. Attached IAM Role to EC2

Went to:

EC2 → Instances → asg-public-ec2 → Actions → Security → Modify IAM role

Selected:

instanceRole

and attached it to the EC2 instance.

10. Restarted CloudWatch Agent

After attaching the IAM role, restarted the CloudWatch Agent:

```
sudo systemctl restart amazon-cloudwatch-agent
```

Then verified the service:

```
sudo systemctl status amazon-cloudwatch-agent
```

Confirmed that the agent was running.

11. Verified Apache Metrics in CloudWatch

Waited for the CloudWatch Agent to publish the metrics.

Opened:

CloudWatch → Metrics → All metrics → Custom namespaces

Found:

Apache/Monitoring

Selected the Apache process metric for the EC2 instance.

The pid_count metric was visible, confirming that CloudWatch was receiving Apache monitoring data.

12. Added Apache Monitoring to Dashboard

Added the Apache pid_count metric to the CloudWatch Dashboard.

Created the widget:

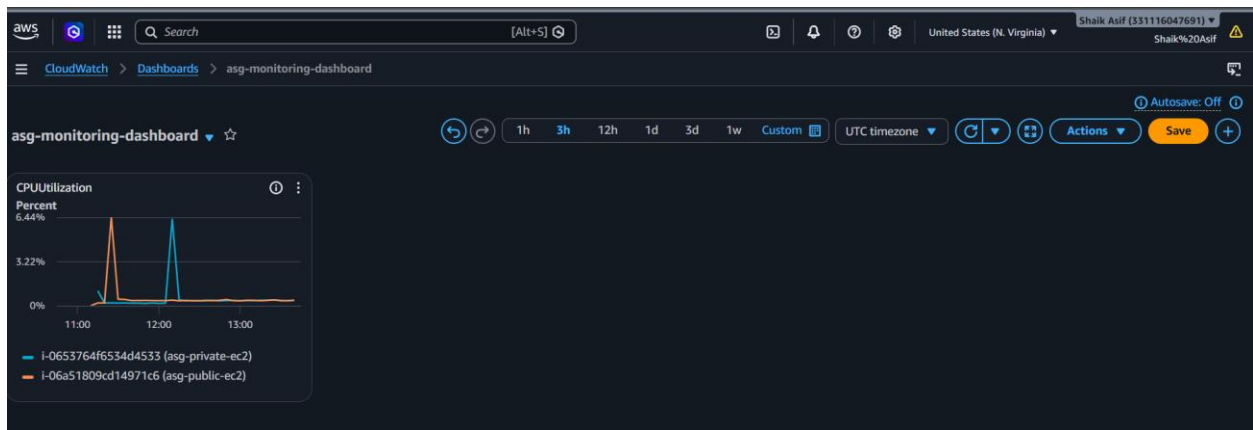
Apache Service Monitoring

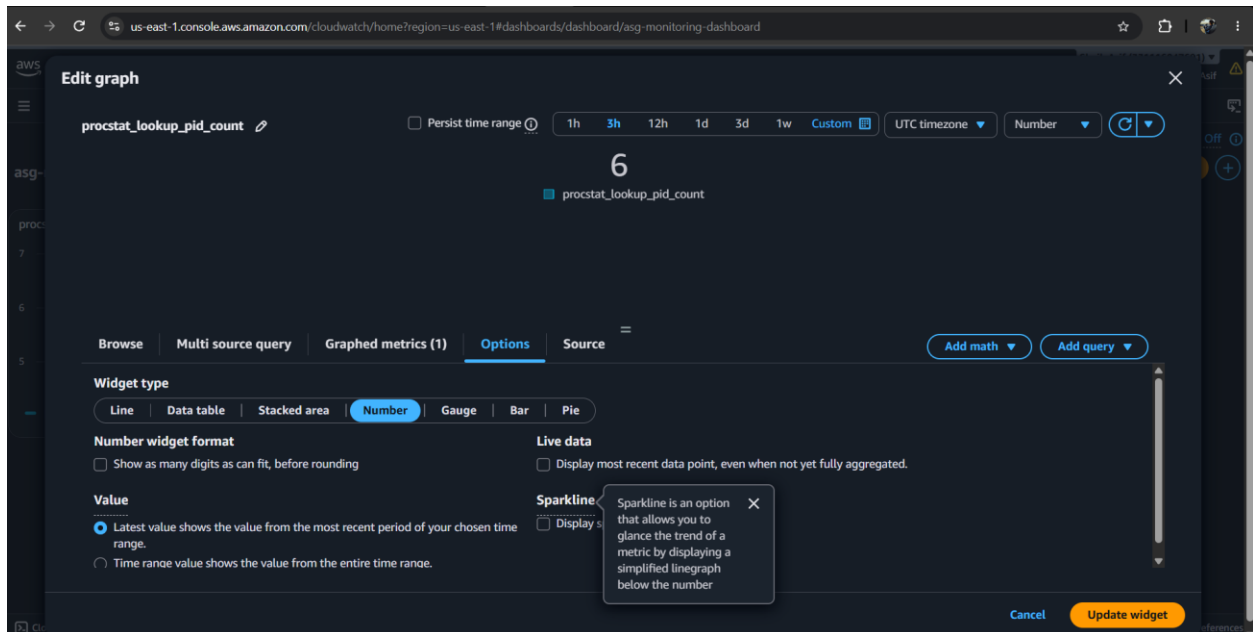
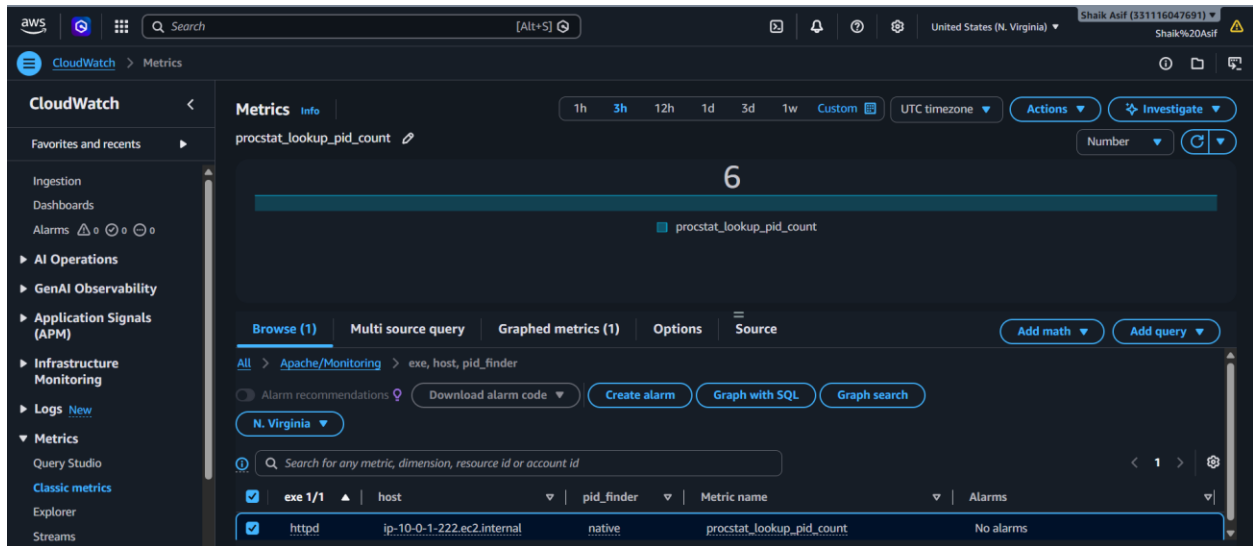
The dashboard now provides monitoring for:

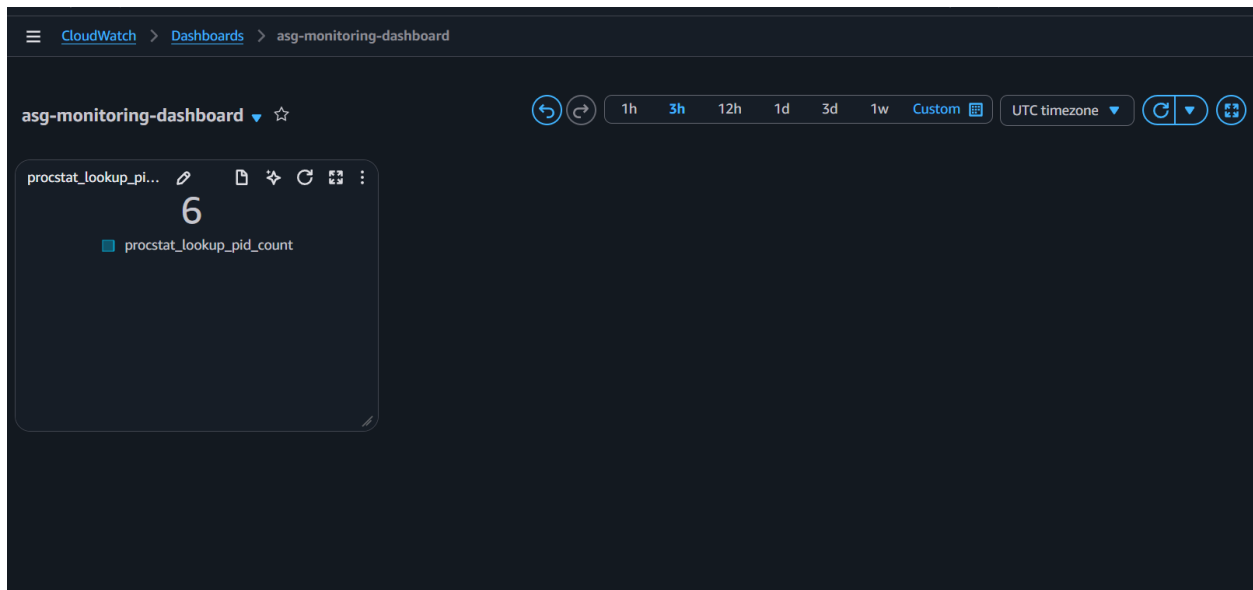
- EC2 CPU Utilization
- Apache Service

Result

Successfully configured CloudWatch monitoring for both the EC2 CPU utilization and Apache service. The CloudWatch Agent is collecting the Apache process pid_count metric and publishing it to the Apache/Monitoring namespace.







Conclusion

The monitoring setup was successfully completed. The CloudWatch Dashboard provides centralized visibility into EC2 CPU usage and Apache service health. The Apache `pid_count` metric can be used to determine whether the Apache process is running, while the CPU metric helps monitor EC2 resource utilization. The IAM role and CloudWatch Agent were configured successfully to enable continuous metric collection and monitoring.